

TRANSCRIBE PROJECT DOCUMENTS

Product Requirements Document (PRD)

Document 01

Product	Transcribe: Kenyan English-Kiswahili code-switching transcriber
Owner	Project lead (sole developer/reviewer for v1)
Status	Draft v1
Version	1.1 · 2026-10-02 02:38 · change log
Related	00 Problem Statement , 02 TRD , 06 Implementation Plan

Contents

1	Vision	1
2	Users and personas	1
3	Product surfaces	2
4	Scope	2
5	Functional requirements	2
6	Non-functional requirements	4
7	Success metrics	4
8	Content and transcription rules (style guide summary)	4
9	Assumptions and risks	5
10	Release plan	5
	Change log	6

1 Vision

Deliver one transcriber that understands how Kenyans actually speak, whether that is English, Kiswahili, or both inside a sentence or a word. It should run on ordinary hardware, and every correction a person makes should feed back into making it better.

2 Users and personas

Persona	Context	Primary need
Corpus reviewer (you, v1)	Has audio + reviewed DOCX transcripts	Turn existing transcripts into verified training segments quickly
Model developer (you, v1)	Learning the fine-tuning process on Kaggle	Reproducible datasets, training runs and evaluation reports to compare models
Transcriber user (v1 internal, later external)	Media, church, research, meeting audio	Upload audio, get an accurate editable transcript, export it

Persona	Context	Primary need
Future annotator (v2)	Additional reviewers	Clear queue, style guide, and no chance of overwriting another person's work

3 Product surfaces

Changed in v1.1: see the change log.

1. **Transcribe:** upload audio or video, get a timestamped transcript, edit it, export it.
2. **Corpus Studio:** import recording + transcript, auto-align, review segments, build frozen datasets.
3. **Models & Evaluation:** register fine-tuned models, run evaluations, compare versions, choose the production model.

All three are served by one local web app (FastAPI backend serving a plain HTML/JavaScript UI) that runs on CPU. Training runs in Kaggle notebooks.

4 Scope

4.1 In scope (v1 = working prototype)

- Transcription of English, Kiswahili, and code-switched speech including intra-word mixing, with readable output (spaces, casing, punctuation).
- Long audio (at least 3 h per file) via VAD-based windowing.
- Corpus Studio with forced alignment of existing transcripts, prioritised review, flags, and language span tags.
- One fine-tuned model (Whisper-small class), trained on Kaggle and served on CPU.
- Evaluation on a frozen, recording-level held-out test set.
- Exports: TXT, SRT, VTT, DOCX, JSON.

4.2 Later (v2+)

- A larger model (Whisper large-v3-turbo or medium via LoRA), an active-learning loop from user corrections, and a CTC comparison model.
- Speaker diarization (who spoke when, as labels only, never identity).
- Domain packs for TV/broadcast, church/sermon and meetings, each with its own evaluation set.
- Multi-user review with roles and second-pass QA.
- Containerised deployment on a small cloud VM or GPU.

4.3 Non-goals

Speaker identification, ethnicity inference, translation, real-time streaming in v1, and languages beyond EN/SW as transcription targets.

5 Functional requirements

Priority: **P0** = required for v1 prototype, **P1** = v1 if time allows, **P2** = later.

5.1 Transcribe

ID	Requirement	P
T-1	Upload WAV/MP3/M4A/MP4/OGG/FLAC up to 3 h; audio is decoded to 16 kHz mono internally	P0
T-2	Transcription runs as a background job with visible progress and ETA	P0
T-3	Output segments carry start/end timestamps and text; word timestamps are available	P0
T-4	Output preserves code-switching as spoken: English words in English spelling, Kiswahili in Kiswahili spelling, hybrid words written as one token	P0
T-5	Non-speech (music, silence, noise) produces no text (hallucination guard)	P0
T-6	In-browser editor: play a segment, edit text, split/merge segments	P0
T-7	Export TXT, SRT, VTT, JSON	P0
T-8	Export DOCX	P1
T-9	Low-confidence words/segments are visually highlighted	P1
T-10	“Promote to corpus”: edited segments can be sent to Corpus Studio as training candidates, only when the recording's consent scope allows it	P1
T-11	Speaker turns (diarization labels Speaker 1/2/...)	P2

5.2 Corpus Studio

ID	Requirement	P
C-1	Import a recording with its DOCX/PDF transcript, domain, and consent scope; the original files are never modified	P0
C-2	Forced alignment of the transcript to the audio produces word timestamps and per-word confidence	P0
C-3	Segmentation: aligned words are grouped into 2–30 s training segments split at natural pauses, never mid-word	P0
C-4	Segments are pre-filled with aligned transcript text; the reviewer verifies rather than types	P0
C-5	Review queue ordered by risk (low alignment confidence, unclear markers, overlap, model disagreement)	P0
C-6	Per-segment flags: unclear, overlap, noisy, music, non-verbatim, reject	P0
C-7	Model hypothesis shown <i>beside</i> the text as a diff; it never replaces reviewer text automatically	P0
C-8	Full revision history per segment; any revision can be restored	P0
C-9	Boundary nudging (± 50 ms / ± 250 ms) with instant replay	P0
C-10	Language span tagging: en, sw, mixed (intra-word), Luo, other, unclear	P1
C-11	Speaker-role filter (interviewer/respondent) configurable per dataset build, not hard-deleted	P0
C-12	Fallback for recordings without transcripts: VAD chunks + model pre-fill	P1
C-13	Keyboard-driven review (approve, flag, play, nudge)	P0

5.3 Datasets, Models & Evaluation

ID	Requirement	P
D-1	Build a dataset snapshot from approved segments using a filter spec; the snapshot is frozen and versioned	P0
D-2	Splits are grouped by recording (and by speaker where known); the test set is frozen across versions	P0
D-3	Export a snapshot as a Hugging Face datasets-compatible package for upload as a private Kaggle Dataset	P0

ID	Requirement	P
D-4	Kaggle notebook trains from a snapshot and outputs model + training log + dataset hash	P0
D-5	Register a model version and convert it to CTranslate2 int8 for CPU serving	P0
D-6	Evaluation report: WER, CER, per-language WER, switch-point error, hallucination rate, RTF; results broken down by domain and flag	P0
D-7	Side-by-side comparison of two model versions on the same test set	P1
D-8	Zero-shot baseline of stock models is recorded before any fine-tuning	P0

6 Non-functional requirements

Area	Requirement
Performance	CPU inference RTF ≤ 0.5 with the v1 model on a 4-core laptop (1 h audio in ≤ 30 min)
Accuracy	See §7
Reliability	Jobs survive app restarts (DB-backed queue); review saves are atomic; no data loss on crash
Privacy	Audio stays local by default. Uploads to Kaggle use private datasets only, and only for recordings whose consent scope includes training
Reproducibility	Every model links to a dataset snapshot hash, base model, config, and notebook version
Portability	Runs on Windows and Linux with Python 3.10–3.11 and ffmpeg
Usability	A trained reviewer verifies 1 h of pre-aligned audio in ≤ 1.5 h

7 Success metrics

Word error rate is measured on the frozen **clean** test subset (single speaker, no overlap, low noise) after text normalisation. The metrics are defined in the [TRD](#).

Metric	Baseline	v1 target	Stretch
WER, clean held-out	measure zero-shot	$\leq 20\%$	$\leq 10\%$ (“90% correct”)
WER, noisy/overlap subset	measure	reported, no gate	$\leq 30\%$
Switch-point error rate	measure	$\leq 1.5\times$ clean WER	$\leq$ clean WER
Hallucination rate (words emitted on non-speech)	measure	< 1 word/min of non-speech	~ 0
CPU RTF	–	≤ 0.5	≤ 0.25
Review throughput	typing from scratch	$\geq 3\times$ faster than typing	$\geq 5\times$

Important: the training data is interviews, while the target deployment includes TV and church audio. A model that scores 10% on interviews can still do much worse on sermons with music and reverb. Each target domain needs a small held-out evaluation set (≥ 1 h each) before any accuracy claim is made for that domain.

8 Content and transcription rules (style guide summary)

The full style guide lives in docs/STYLE_GUIDE.md (created in Phase 0).

- **Verbatim:** transcribe what was said, including code-switching and repetitions that carry meaning. Pure fillers (*eh*, *umm*) are dropped by default. This is a documented choice, applied the same way everywhere.
- **Hybrid words:** one token, no hyphen, with the English stem in English spelling: *nimedownload*, *ame-*

organize. The normaliser strips hyphens anyway, so legacy hyphenated text still scores correctly.

- **Numbers:** source documents use digits. Training targets use words in the language actually spoken (*sita* vs *six*). Years default to English (*twenty nineteen*), and labels use English title case (*Level Five*). The aligner picks between the English and Kiswahili candidates by audio score. See [STYLE_GUIDE §4](#).
- **Language marking:** italic = Kiswahili in every DOCX; hybrid words are one word with only the Kiswahili part italic (*nimedownload* → tagged mixed).
- **Unclear speech:** [unclear] in the source transcript. These segments are excluded from training unless the reviewer resolves them.
- **Non-EN/SW words** (e.g. Dholuo): transcribe them as heard and tag the span `luo/other`.
- **Non-verbatim source passages** (the transcriber summarised or skipped text for meaning): flag `non_verbatim`. The segment is excluded from training until it is corrected to what was actually said.

9 Assumptions and risks

Risk	Impact	Mitigation
Source transcripts are partly non-verbatim	The model learns to paraphrase or hallucinate	Alignment confidence flags mismatches; a <code>non_verbatim</code> flag excludes them
Domain shift (interviews → TV/church)	Poor real-world accuracy	Per-domain evaluation sets; add domain data in v2
Kaggle GPU quota and session limits	Slow iteration	Whisper-small first; resumable checkpoints; small experiments
Whisper translating instead of transcribing code-switched speech	Wrong language in output	Fine-tune with a fixed language token on code-switched data; switch-point metric
Hallucination on music/silence (churches)	Garbage text	VAD gating, non-speech training examples, decode filters
Consent limits on the research audio	Legal/ethical exposure	<code>consent_scope</code> per recording enforced at dataset build and Kaggle export
Single reviewer	Throughput and bias	Risk-ordered queue; add second-pass QA in v2

10 Release plan

Milestone	Contents
M0 Foundations	Bug fixes, DB, style guide, grouped splits
M1 Corpus v1	Alignment + review UI v2, first dataset snapshot, zero-shot baselines
M2 Model v1	Whisper-small fine-tuned on Kaggle, evaluation report
M3 Prototype	Transcribe surface on CPU with exports, v1 release
M4 Loop	Corrections → corpus, model v2, domain evaluation sets

See [06 Implementation Plan](#) for tasks and acceptance criteria.

Change log

Times are East Africa Time (EAT, UTC+3). Each entry names the affected section, quotes what it said before and summarises what it says now, and links to the previous text: on GitHub at the exact commit and lines, and as an archived PDF of the previous version.

Version	Date (EAT)	Commit	Summary
1.0	2026-09-24 00:36	053e09d	First LaTeX version (converted from the Markdown draft of 2026-09-23).
1.1	2026-10-02 02:38	this commit	UI technology: plain HTML/JavaScript instead of Gradio (see UI/UX v1.1).

v1.1 · §3 Product surfaces

Previous text: [v1.0 source, line 48](#) · [v1.0 PDF](#)

Before. “All three are served by one local web app (FastAPI backend + Gradio UI) that runs on CPU.”

Now. “...(FastAPI backend serving a plain HTML/JavaScript UI) ...”. Gradio could not capture F1–F4 or control playback speed.